



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

Lenguaje y compiladores trimestre 2026-II

fmarquez@e.uneg.edu.ve

Tema 1 (Introducción a los Lenguajes y Compiladores)

- 1.1. Aportes teóricos fundamentales de la Cs. de la computación.
- 1.2. Diagramas T.
- 1.3. Metacompiladores.
- 1.4. Decompiladores.
- 1.5. Interpretes.

Evaluación: 10 ptos defensa y 5 informe de investigación y entregables. (ver sección 9)



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

Contenido.

1. Introducción.....	3
2. Bases teóricas logica matemática y lenguajes formales.....	3
3. Arquitectura del compilador:fases clásicas y optimización moderna.....	5
3.1. Análisis Léxico.....	5
3.2. Análisis Sintáctico.....	5
3.3. Análisis Semántico y Verificación de Tipos.....	6
3.4. Generación de Código Intermedio y Optimización.....	7
4.- Diagramas T, metacompiladores, y bootstrapping.....	8
5.- Interpretes. JIT y compilación bajo demanda.....	9
6. Decompiladores y el rol en la ingeniería inversa y seguridad.....	9
7. Aplicaciones y tópicos innovadores para la investigación.....	10
7.1. Validación de Infraestructura como Código (IaC).....	11
7.2. Lenguajes de Dominio Específico (DSL).....	11
7.3. Formateadores, Linters y Análisis Estático.....	11
7.4. Procesamiento de Lenguaje Natural y Tokenización.....	11
7.5. Hipercomputación y Reconocimiento de Lenguajes.....	12
7.6. Compilación Asistida por IA y LLMs.....	12
7.7. Compiladores para Ecosistemas Emergentes.....	12
7.8. Propuesta de Tópicos para el TEMA 1.....	13
8. Conclusiones.....	14
9.- Consideraciones generales para la evaluación.....	16



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

1. Introducción.

Imaginar un compilador como una “caja mágica” que transforma texto en instrucciones ejecutables es una simplificación pedagógica útil, pero insuficiente para comprender su verdadera naturaleza. En esencia, un compilador es un sistema formal de traducción que opera sobre reglas matemáticas bien definidas, puentes entre la expresividad humana y la determinística de la máquina. Como señalaban Aho, Lam, Sethi y Ullman (2006), la construcción de compiladores constituye uno de los campos más maduros de la informática, cuyos principios trascienden la mera traducción de lenguajes y se proyectan hacia el diseño de protocolos, validación de esquemas, análisis de seguridad y estructuración de sistemas complejos.

El objetivo de este documento es parafrasear, profundizar y expandir los conceptos introductorios del Tema 1 del curso *Lenguajes y Compiladores*, incorporando avances teóricos, arquitectónicos y metodológicos vigentes en 2026. Se busca ofrecer una base conceptual sólida que sirva tanto para la formación académica como para la elaboración de informes de investigación y defensas grabadas, alineados con los tópicos propuestos por la coordinación. Además, se integran perspectivas modernas como la compilación asistida por IA, los ecosistemas WASM, la verificación formal, la infraestructura como código y la hipercomputación aplicada al reconocimiento de lenguajes, garantizando que el contenido refleje la evolución del campo sin perder el rigor formal que lo sustenta.

Finalmente, este material está diseñado para ser adaptable: puede emplearse como guía de estudio, como marco teórico para trabajos grupales o como base para la elaboración de los informes académicos que exige la asignatura. Su extensión y profundidad buscan cubrir aproximadamente diez páginas en formato académico estándar, cumpliendo con los requisitos de citación APA, estructura lógica y aplicabilidad práctica señalados en la planificación del curso.

2. Bases teóricas logica matemática y lenguajes formales.

La teoría de compiladores no surge en un vacío tecnológico; es la culminación de siglos de desarrollo en lógica, lingüística formal y matemáticas discretas. Los aportes de Gödel, Turing, Church y Chomsky establecieron los límites y posibilidades de lo que puede ser computado, reconocido o generado mediante



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

reglas formales. La tesis de Church-Turing demostró que cualquier función computable puede ser expresada mediante una máquina abstracta con memoria ilimitada y un conjunto finito de reglas de transición. Este modelo, lejos de ser una reliquia histórica, sigue siendo la base teórica sobre la que se diseñan los analizadores sintácticos, las gramáticas generativas y los autómatas que componen las fases iniciales de un compilador.

La jerarquía de Chomsky, por su parte, clasificó los lenguajes formales en cuatro niveles (tipo 0 a tipo 3), estableciendo una correspondencia directa con las capacidades de reconocimiento de distintas máquinas abstractas. Los lenguajes regulares (tipo 3) son reconocidos por autómatas finitos y constituyen la base del análisis léxico. Los lenguajes independientes del contexto (tipo 2) son procesados por autómatas de pila y sustentan el análisis sintáctico. Los lenguajes sensibles al contexto y recursivamente enumerables abren la puerta a la semántica estática, la verificación de tipos y, en casos avanzados, a la optimización interprocedimental.

En la actualidad, esta jerarquía no solo se estudia como teoría pura, sino que se materializa en herramientas como ANTLR, Bison, Flex y Lexer, que automatizan la generación de analizadores a partir de gramáticas declarativas. Asimismo, la integración de técnicas de procesamiento de lenguaje natural (NLP) con lenguajes formales ha permitido crear interfaces híbridas donde el código se genera o valida mediante modelos probabilísticos, sin perder la garantía de corrección estructural que solo las reglas formales pueden ofrecer.

RETO SOLO PARA CURIOSOS: 1) *cual es poder real de la máquina de turin y que fue capaz de resolver, ¿se construyó algún prototipo de esta máquina? y por otro lado que relación tiene como validación desde las ciencias de la computación los LLM y la prueba de turin.*

2) *descubre los tipos de problemas P, NP y NP-completo ,su definición, interpretación práctica y que ejemplo de problemas están clasificado en esos tipos.*



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

3. Arquitectura del compilador:fases clásicas y optimización moderna.

Un compilador tradicional se estructura en fases secuenciales, cada una con responsabilidades bien delimitadas y representables mediante grafos de dependencia y flujos de datos. Aunque la división clásica sigue vigente, la arquitectura moderna ha evolucionado hacia diseños modulares, reutilizables y orientados a representaciones intermedias universales.

3.1. Análisis Léxico

El análisis léxico transforma el flujo de caracteres en tokens (lo que usted como experto en LyC llamará tokenización) significativos mediante patrones estructurados. Teóricamente, se fundamenta en autómatas finitos determinísticos (AFD) y no determinísticos (AFN), que modelan transiciones de estado para reconocer secuencias válidas con complejidad lineal. Las expresiones regulares operan como especificaciones declarativas de estos patrones, compilándose internamente a AFD minimizados que garantizan reconocimiento eficiente sin backtracking excesivo. Herramientas como Flex, ANTLR y RE2 automatizan esta conversión, asegurando robustez y mantenibilidad industrial. Implementar un lexer manualmente sin estas utilidades exige codificar máquinas de estado explícitas, gestionar casos límite y validar exhaustivamente, lo que eleva la propensión a errores y costos de desarrollo, aunque consolida el dominio algorítmico. Actualmente, se complementan con PEG, combinadores funcionales, lexers zero-copy y procesamiento por streaming. Además, emergen tokenizadores híbridos que integran modelos probabilísticos para entradas semiestructuradas, sin sacrificar la garantía formal. Esta evolución mantiene al análisis léxico como la barrera crítica de validación antes del parsing.

3.2. Análisis Sintáctico

El parser valida la estructura gramatical del programa. Tradicionalmente se empleaban parsers LL y LR, pero la tendencia actual favorece parsers PEG y GLR (Generalized LR), que manejan ambigüedades y lenguajes no deterministas con mayor flexibilidad. Frameworks como Tree-sitter han popularizado el parsing incremental y tolerante a errores, esencial para editores modernos y análisis estático en tiempo real.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

En este punto el clic de la etapa es tener una buena abstracción del lenguaje expresado en términos de una gramática, en los lenguajes de programación el dominio de las gramáticas de libre contexto es determinante.

El análisis sintáctico valida la estructura jerárquica del código mediante reglas de derivación formal. Los analizadores LL (descendente, derivación por la izquierda) y LR (ascendente, derivación por la derecha inversa) son deterministas y óptimos para lenguajes no ambiguos. Las GLC (Gramáticas Libres de Contexto) constituyen el marco matemático que estos parsers procesan. PEG (Gramáticas de Expresiones de Parsing) elimina ambigüedades mediante elección priorizada y backtracking determinista, mientras GLR (LR Generalizado) paraleliza múltiples rutas de análisis para soportar GLC ambiguas sin perder cobertura.

RETO NIVEL DIOS: No te quedes con la dudas en tu tiempo libre adelanta tu entrenamiento en LyC e investiga que son gramáticas, como se derivan y visualiza cuando una GLC es ambigua y cuando es recursiva lo necesitaras para temas posteriores.

3.3. Análisis Semántico y Verificación de Tipos

El análisis semántico y la verificación de tipos constituyen la fase donde se valida la coherencia lógica del programa más allá de su estructura gramatical. Aquí se emplean técnicas como la construcción y recorrido de Árboles de Sintaxis Abstracta (AST), esto es esencial, los cuales aplican directamente en este proceso. Tras el análisis sintáctico, el AST representa la jerarquía del código sin detalles léxicos superfluos, facilitando la validación semántica. Mediante recorridos en profundidad, se ejecutan operaciones clave: construcción de tablas de símbolos para resolver alcances y vinculaciones, inferencia y unificación de tipos (Hindley-Milner), y análisis de flujo de datos para detectar variables no inicializadas o coerciones inválidas. Es por eso que en lenguajes modernos, se complementa con verificación de propiedades (*ownership*, linealidad) y tipado gradual, permitiendo interoperabilidad entre sistemas estáticos y



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

dinámicos. Estas técnicas garantizan que las operaciones sean seguras y significativas antes de la generación de código intermedio. El AST no solo aplica, sino que es la estructura fundamental que habilita el análisis semántico eficiente, ya que preserva la intención del programador en un formato anidado y optimizado para transformaciones, optimizaciones y verificaciones formales posteriores.

RETO NIVEL DIOS: No te quedes con la dudas en tu tiempo libre adelanta tu entrenamiento en LyC e investiga que son los AST y su implicación en el tipado del código.

3.4. Generación de Código Intermedio y Optimización

La representación intermedia (IR) actúa como puente entre el lenguaje fuente y el código máquina. LLVM IR, MLIR (Multi-Level Intermediate Representation) y WASM Text Format son ejemplos de IR modernas que permiten optimizaciones independientes de la arquitectura. La optimización moderna incluye:

- Eliminación de código muerto y propagación de constantes
- Desenrollado de bucles y vectorización automática (SIMD)
- Inlining agresivo y especialización por perfil (PGO)
- Optimizaciones guiadas por IA que predicen patrones de ejecución y ajustan la generación de código en tiempo de compilación

El backend traduce la IR optimizada a instrucciones nativas o bytecode, gestionando registros, asignación de memoria y llamadas ABI. En 2026, la tendencia es hacia backends polyglot y multi-objetivo, capaces de generar código para x86_64, ARM64, RISC-V, GPUs (PTX, SPIR-V) y entornos de ejecución aislados (WASM, SGX).



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

4.- Diagramas T, metacompiladores, y bootstrapping.

Los diagramas T, introducidos en los años 60, son una notación visual que representa la relación entre lenguajes fuente, lenguajes objetivo y el lenguaje de implementación de una herramienta de traducción. Su forma de "T" simboliza cómo un compilador escrito en un lenguaje I transforma programas en lenguaje S a código en lenguaje O. Esta abstracción fue crucial para comprender el proceso de *bootstrapping*: la capacidad de un compilador de compilar su propio código fuente, creando así un ciclo de autoperpetuación que independiza al ecosistema de herramientas externas.

Los metacompiladores generalizan este concepto: son programas que, a partir de una descripción formal de un lenguaje (gramática, semántica, reglas de generación), producen automáticamente un compilador o intérprete. Herramientas como Lex/Yacc, ANTLR, OCamllex/Menhir y los generadores de parsers en Rust o Haskell son ejemplos modernos de metacompiladores parciales. En la actualidad, la meta-compilación se ha expandido hacia:

- Generación de validadores de esquemas (JSON Schema, OpenAPI, Protobuf)
- Síntesis de transpiladores (*transcompiler*) entre lenguajes (TypeScript -> JS, Kotlin -> JVM, C++ -> WASM)
- Frameworks de compilación declarativa (MLIR dialects, Terraform providers)

El bootstrapping sigue siendo un hito pedagógico y técnico. Compilar un compilador desde cero exige dominar las fases de análisis, gestionar la dependencia circular y validar la corrección del generador. Proyectos como cc65, TinyCC, Vlang y Zig demuestran que el bootstrapping no es solo un ejercicio académico, sino una práctica de ingeniería que garantiza trazabilidad, auditoría y soberanía tecnológica.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

5.- Interpretes. JIT y compilación bajo demanda.

La dicotomía clásica entre compiladores e intérpretes ha dado paso a arquitecturas híbridas. Un intérprete ejecuta instrucciones línea por línea, ofreciendo flexibilidad y retroalimentación inmediata, pero sacrificando rendimiento. Un compilador genera código nativo de antemano, maximizando velocidad pero requiriendo ciclos de desarrollo más largos. La compilación JIT (Just-In-Time) fusiona ambas aproximaciones: analiza el código en tiempo de ejecución, perfila su comportamiento y genera código nativo optimizado solo para las rutas críticas.

Motores como V8 (JavaScript/TypeScript), HotSpot (JVM), PyPy (Python) y .NET CLR emplean JIT de múltiples niveles (Tier 0 a Tier 3), donde el código se interpreta inicialmente, se compila con optimizaciones básicas y, finalmente, se recompila con perfiles de ejecución reales. Esta estrategia ha permitido que lenguajes dinámicos compitan en rendimiento con lenguajes estáticos en dominios específicos.

Un punto que considero interesante, es que la interpretación ha evolucionado a modelos de sandboxing y verificación formal. WebAssembly (WASM), por ejemplo, se ejecuta en un runtime seguro, con validación de bytecode previa a la ejecución, permitiendo llevar aplicaciones de alto rendimiento al navegador sin exponer el sistema host. En entornos de edge computing y IoT, los intérpretes ligeros (como MicroPython o LuaJIT) ofrecen un equilibrio entre consumo de recursos y capacidad de scripting, demostrando que la interpretación no es un modelo obsoleto, sino una arquitectura adaptativa.

6. Decompiladores y el rol en la ingeniería inversa y seguridad.

Los decompiladores realizan el proceso inverso al de un compilador: transforman código binario o bytecode en una representación de alto nivel legible por humanos. Aunque históricamente se asociaron con fines de análisis de malware o ingeniería inversa comercial, hoy son herramientas esenciales en:



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA

LENGUAJES Y COMPILADORES

Ing. Félix Márquez, Msc

- Auditoría de software propietario y legado
- Recuperación de código fuente perdido
- Validación de compiladores cruzados y toolchains
- Detección de vulnerabilidades en binarios sin acceso al repositorio

Herramientas como Ghidra, IDA Pro, RetDec y Binary Ninja emplean técnicas de análisis de control de flujo, reconstrucción de estructuras de datos, inferencia de tipos y patrones de compilación para generar pseudocódigo estructurado. En 2026, la integración de modelos de lenguaje (LLMs) con motores de desensamblado ha permitido mejorar la legibilidad del código decompilado, aunque con advertencias críticas: la descompilación no es una transformación biyectiva. Información semántica, nombres de variables y comentarios se pierden irremediabilmente durante la compilación, por lo que el resultado siempre es una aproximación, no una réplica exacta.

En el ámbito de la ciberseguridad, los decompiladores son componentes clave en la cadena de análisis de vulnerabilidades (CVE), detección de supply-chain attacks y verificación de integridad de firmwares. Su uso ético y regulado está cada vez más normalizado en auditorías oficiales, respuestas a incidentes y desarrollo de parches de mitigación.

RETO NIVEL DIOS: demuestra si eres capaz de crackear un programa, pídele a un integrante de tu equipo que cree un programa que para ingresar solicite una contraseña. Decompílalo y miras la clave que le colocó y crea una nueva versión de ejecutable con una nueva contraseña.

7. Aplicaciones y tópicos innovadores para la investigación.

El conocimiento de compiladores ya no se limita a la construcción de lenguajes de programación. Sus técnicas se han transversalizado a múltiples dominios, lo que justifica la incorporación de tópicos innovadores en la investigación académica:



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

7.1. Validación de Infraestructura como Código (IaC)

Archivos YAML/JSON en Terraform, Kubernetes, Ansible o GitHub Actions requieren validación léxica, sintáctica y semántica estricta. Un puerto inválido, una ruta inexistente o una dependencia circular pueden colapsar entornos productivos. La aplicación de gramáticas formales y analizadores semánticos garantiza la corrección antes del despliegue.

7.2. Lenguajes de Dominio Específico (DSL).

Desde consultas SQL hasta fórmulas en hojas de cálculo, los DSL son mini-lenguajes con sintaxis y semántica restringidas. Su diseño implica análisis léxico-sintáctico, resolución de contexto y generación de ejecutores seguros. En 2026, los DSLs se emplean en contratos inteligentes (Solidity, Move), pipelines de datos (dbt, SQLMesh) y configuración de modelos de IA (LangChain, DSPy).

7.3. Formateadores, Linters y Análisis Estático.

Prettier, ESLint, Clang-Tidy y SonarQube utilizan reglas sintácticas y semánticas para detectar anti-patrones, vulnerabilidades (inyecciones SQL, XSS, deserialización insegura) y inconsistencias de estilo. La integración con CI/CD automatiza la corrección temprana, reduciendo costos de depuración.

7.4. Procesamiento de Lenguaje Natural y Tokenización.

La tokenización en NLP (WordPiece, BPE, SentencePiece) comparte fundamentos con el análisis léxico de compiladores. La diferencia radica en que los tokens en NLP son probabilísticos y contextuales, mientras que en lenguajes formales son deterministas. No obstante, arquitecturas híbridas (como parsers con fallback a LLMs) están emergiendo para validar entradas naturales y transformarlas en DSLs ejecutables.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

7.5. Hipercomputación y Reconocimiento de Lenguajes.

Más allá de la máquina de Turing, modelos teóricos como las máquinas oráculo, la computación analógica infinita o los sistemas cuánticos topológicos exploran límites de reconocimiento de lenguajes recursivamente enumerables. Aunque aún teóricos, estos modelos inspiran algoritmos de parsing aproximado, verificación probabilística y resolución de problemas indecidibles en dominios restringidos.

7.6. Compilación Asistida por IA y LLMs.

Modelos como CodeLlama, StarCoder y Gemini Code pueden generar código binario (x64, ARM) a partir de prompts en lenguaje natural. Sin embargo, la generación directa de binarios sin verificación formal es riesgosa. La tendencia es usar LLMs como asistentes de síntesis, mientras el compilador tradicional valida, optimiza y garantiza corrección. Proyectos como llm-compiler y Aider integran ambos mundos bajo principios de verificación cruzada.

7.7. Compiladores para Ecosistemas Emergentes.

- Quantum Compilers: Qiskit, Cirq y TKET traducen circuitos cuánticos abstractos a pulsos de hardware, optimizando profundidad de circuito y corrección de errores.
- WebAssembly System Interface (WASI): Permite ejecutar binarios compilados fuera del navegador, con sandboxing y acceso controlado a recursos.
- Compiladores de Modelos de IA: TVM, MLIR y XLA optimizan grafos de tensores para GPUs, TPUs y NPUs, aplicando transformaciones propias de compiladores tradicionales a dominios de machine learning.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

7.8. Propuesta de Tópicos para el **TEMA 1**.

A continuación se listan unos series de tópicos para que cada grupo selecciones uno, lo investigue y discuta de forma critica con el equipo, crear el informe de investigación y cada integrante defensa de un punto en particular o resumen del tema, los términos de la defensa es individual y cada participante tiene que dar cuenta que domina el tema al final se entrega un único video de defensa de los integrantes del grupo:

- 1.- Comparativa de AFD vs AFN en el análisis léxico de lenguajes modernos.
- 2.- Evolución de los diagramas T hacia arquitecturas de meta-programación y generación de código asistida por IA.
- 3.- Fases de un compilador moderno: de LLVM a MLIR, ¿es el IR el nuevo lenguaje universal?
- 4.- Tokenización en NLP vs. Tokenización en lenguajes formales: convergencias y divergencias.
- 5.- Autómatas finitos en la validación de contratos inteligentes y pipelines de datos.
- 6.- Diálogo teórico entre Gödel y Turing: indecidibilidad, límites de la compilación y verificación formal.
- 7.- Expresiones regulares modernas: RE2, PEG y límites de reconocimiento léxico.
- 8.- Generación de binarios Linux x64 a partir de prompts en español: riesgos, validación y ética.
- 9.- Gramáticas generativas y Jerarquía de Chomsky en la era de los LLMs.
- 10.- GLC con notaciones BNF/EBNF/LBNF: generación de FEN y PGN para motores de ajedrez.
- 11.- Teoría de decompiladores aplicados a firmware IoT y análisis de malware.
- 12.- Arquitectura de Transformers en el reconocimiento de sintaxis semiestructurada.
- 13.- Tokenización conceptual y práctica: implicaciones en compiladores, parsers y validadores.
- 14.- Diferencias y similitudes entre decompiladores, metacompiladores e interprete y su vinculación en la actualidad con problemas reales.
- 15.- Mecanismos que interactúan en WASM y su importancia en tiempo de respuesta de las API.
- 16.- Evolución de los diagramas T.
- 17.- Explicar las fases de un compilador, tendencias.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

- 18.- Técnicas de procesamiento de lenguaje natural, evolución.
- 19.- Autómata finitos determinísticos (AFD) y Atómatas finitos no Determinismos (AFN).
- 20.- Diferencias y similitudes en los aportes de Godel y Turing, luego plantee un dialogo entre godel y turing sobre la base de sus teorías o postura que presenten sus puntos de vista acerca de los los compiladores. ¿Que es un compilador?, ¿como secan los computadores del futuro? ¿Las máquinas basadas en cintas se pueden usar para desarrollo de compiladores máquina de turin?, la máquina de turin fue de verdad utilizada para descubrir el código nazi (máquina enigma)?, existe un poder de reconocimiento de lenguajes superior a la máquina de turin?.
- 21.- Expresiones regulares, su capacidad de reconocimiento léxico y vinculación con los autómatas.
- 22.- Utilización de LLM para la creación de código binario linux x64 partiendo de un prompt en español
- 23.- Ejemplo de GLC basada en notación Backus- Naur, EBNF y LBNF, explique mediante aplicación de producciones para generar un lenguaje en particular, muestre ejemplos para generar lenguaje FEM y PGN en el ámbito de ajedrez.
- 24.- Explique la arquitectura de los Transformers en el reconocimiento de NLP.
- 25.- Explique diferencias entre tokenización (NPL y lenguajes formales) desde lo conceptual y mediante ejemplos al menos 3 ejemplos y su implicación con los compiladores si existe?
- 26.- Herramientas para el proceso de cracking y la importancia de los depompiladores en la ingeniería inversa de software
- 27.- Implicaciones de las gramáticas libres de contexto (GLC) y su ubicación dentro de la jerarquía de Noan Chonsky dentro de las gramáticas generativas.

8. Conclusiones.

El estudio de los compiladores no es un ejercicio académico aislado, sino una disciplina transversal que conecta lógica, lingüística, ingeniería de software y ciberseguridad. Las fases de análisis, transformación y



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA

LENGUAJES Y COMPILADORES

Ing. Félix Márquez, Msc

generación de código modelan la forma en que estructuramos, validamos y optimizamos cualquier sistema basado en reglas formales. Desde la validación de archivos de configuración hasta la generación de circuitos cuánticos, las técnicas de compilación se han convertido en infraestructura invisible pero crítica del desarrollo tecnológico contemporáneo.

La evolución desde los diagramas T y los metacompiladores clásicos hasta los ecosistemas modernos de LLVM, MLIR, WASM y compilación asistida por IA demuestra que el campo no está estancado, sino en constante renovación. Los decompiladores, intérpretes JIT y validadores semánticos ya no son herramientas de nicho, sino componentes esenciales en la cadena de valor del software moderno. Asimismo, la integración de NLP, hipercomputación teórica y verificación formal abre nuevas fronteras donde la ambigüedad natural y el rigor formal pueden coexistir bajo arquitecturas híbridas.

Para los estudiantes de Ingeniería en Informática, dominar estos conceptos no solo significa aprobar una asignatura, sino adquirir una mentalidad analítica capaz de descomponer problemas complejos, diseñar lenguajes de comunicación efectivos y construir sistemas robustos, auditables y eficientes. La defensa grabada y el informe de investigación no son meros requisitos administrativos, sino oportunidades para demostrar dominio técnico, capacidad de síntesis y visión crítica frente a tecnologías en rápida evolución.

La programación es un hermoso proceso de cognitivo que no es solo decirle a una máquina qué hacer, es aprender a comunicarte con claridad en un universo que no perdona errores. El compilador, en ese sentido, es el primer maestro formal de esa disciplina.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

9.- Consideraciones generales para la evaluación.

La evaluación de los temas se desarrollará mediante trabajo colaborativo y entregas formales. Para garantizar trazabilidad, orden académico y un desarrollo profesional coherente, se establecen los siguientes lineamientos obligatorios:

- Conformación de Equipos: Cada grupo estará integrado por 4 estudiantes exactos. Esta conformación será fija para todos los temas del semestre, promoviendo continuidad, responsabilidad compartida y seguimiento.
- Registro Oficial del Grupo: Previo a la entrega de cualquier asignación, el líder deberá formalizar la conformación del equipo indicando:
 - 1.Nombre del grupo (identificación única).
 - 2.Eslogan o lema de trabajo (refleja el enfoque metodológico del equipo).
 3. Cédula y Nombres completos de cada integrante, especificando claramente la sección a la que pertenece.
 - 4.Tópico seleccionado de la Sección 7.8 (*Propuesta de Tópicos para Defensas Grupales*). Este tópico será el eje central de investigación y defensa para el Tema 1.
- Selección de tópicos para el **TEMA** .

No se pueden repetir los tópicos, los tópicos serán aceptados por orden de llegada del correo enviado por el líder, se confirmará por correo si fue aceptado o por el contrario debe cambiar el tópico a trabajar para este tema.
- Gestión de Repositorio Digital:El líder del grupo será responsable de crear y mantener un repositorio académico (GitHub, GitLab). Este espacio deberá contener una carpeta o entrada diferenciada por cada tema (/Tema1, /Tema2, etc.), donde se alojarán las versiones finales de los entregables, material de apoyo, bitácoras de ensayo-error y enlaces de referencia.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJES Y COMPILADORES
Ing. Félix Márquez, Msc

- Entregables y Envío Oficial, cada tema requerirá los siguientes entregables:
 - Informe de investigación grupal con normas APA estrictas (citaciones y referencias obligatorias).
 - Video de defensa compilado en un único archivo, subido a YouTube. El enlace debe quedar registrado tanto en el repositorio temático del grupo como en el cuerpo del correo de entrega.
 - Presentación o material de apoyo.
- Protocolo de Envío por Correo
 1. El líder será el único autorizado para enviar los correos con los resultados.
 2. El asunto del correo debe seguir el formato: [Tema X] - Entrega [Nombre del Grupo].
 3. En el cuerpo del mensaje se incluirá: identificación completa del grupo (nombre, eslogan, integrantes con secciones), enlace al repositorio temático, enlace directo al video de YouTube y el informe adjunto.
 4. Se valorará la precisión en la identificación y la organización de los archivos; envíos incompletos o desestructurados podrán ser devueltos sin calificación.

Fecha de entrega próximo Miércoles 13/05/2025